# 动态规划知识点整理

## 目录


**1 动态规划的原理** **2**

1.1 从递归到动态规划：青蛙跳台阶 . . . . . 2

1.2 动态规划的特点 . . . . . 3

1.3 使用动态规划的条件 . . . . . 3

1.3.1 最优子结构 . . . . . 3

1.3.2 重叠子问题 . . . . . 3

1.4 动态规划算法的设计步骤 . . . . . 3

**2 矩阵链乘法问题** **4**

2.1 问题定义 . . . . . 4

2.2 解空间规模 . . . . . 4

2.3 最优子结构 . . . . . 4

2.4 状态转移方程 . . . . . 5

2.5 自底向上计算 . . . . . 5

2.6 复杂度 . . . . . 6

**3 最长公共子序列问题** **7**

3.1 基本概念 . . . . . 7

3.2 最优子结构 . . . . . 7

3.3 递归方程 . . . . . 7

3.4 算法 . . . . . 8

3.5 复杂度 . . . . . 8

**4 最长上升子序列** **9**

**5 0-1 背包问题** **9**

5.1 问题定义 . . . . . 9

5.2 状态定义与最优子结构 . . . . . 10

5.3 递推方程 . . . . . 10

5.4 构造最优解 . . . . . 10

5.5 算法与复杂度 . . . . . 11
5.6 PPT 例题 . . . . . 11

**6 完全背包问题 12**

**7 状态压缩 12**

**8 最优二叉搜索树 13**
8.1 问题定义 . . . . . 13
8.2 动态规划递推 . . . . . 13
8.3 算法 . . . . . 14

**9 期末相关例题 14**
9.1 例题 1：动态规划概念判断 . . . . . 14
9.2 例题 2：01 背包填空 . . . . . 15
9.3 例题 3：最长公共子串 . . . . . 16
9.4 例题 4：最长回文子串 . . . . . 17
9.5 例题 5：序列计数 . . . . . 19

# 1 动态规划的原理

## 1.1 从递归到动态规划：青蛙跳台阶

一只青蛙一次可以跳 1 级台阶，也可以跳 2 级台阶。若要求跳上第 $n$ 级台阶的方法数，记为 $f(n)$。

要到达第 $n$ 级台阶，最后一步只有两种来源：

- 先到达第 $n-1$ 级，再跳 1 级；
- 先到达第 $n-2$ 级，再跳 2 级。

因此有递推关系

$$f(n) = f(n-1) + f(n-2).$$

边界条件为

$$f(1) = 1, \qquad f(2) = 2.$$

若直接递归：

```
FROG-REC(n)
    if n == 1 return 1
    if n == 2 return 2
    return FROG-REC(n - 1) + FROG-REC(n - 2)
```

递归会反复计算相同子问题。例如 $f(n-2)$ 会在 $f(n-1)$ 与 $f(n)$ 的展开中多次出现。递归树规模随 $n$ 指数增长，因此时间复杂度为

$$O(2^n).$$

动态规划按从小到大的顺序计算并保存结果：

```
FROG-DP(n)
    f[1] = 1
    f[2] = 2
    for i = 3 to n
        f[i] = f[i - 1] + f[i - 2]
    return f[n]
```

每个 $f(i)$ 只计算一次，因此时间复杂度为

$$O(n),$$

若只保留前两项，空间复杂度可由 $O(n)$ 优化为 $O(1)$。

## 1.2 动态规划的特点

动态规划是一种算法设计范式或思想，不是某一个具体算法。它的核心特点是：

- 把原始问题划分成一系列子问题；
- 每个子问题只求解一次；
- 将子问题结果保存在表中，以后需要时直接访问，不重复计算；
- 通常自底向上地计算；
- 适用于具有重叠子问题结构的问题：问题可分解为多个相关子问题，且子问题的解会被重复使用。动态规划不只用于优化问题，也广泛用于计数、判定和可行性问题。

## 1.3 使用动态规划的条件

### 1.3.1 最优子结构

当一个问题的最优解包含了子问题的最优解时，称该问题具有最优子结构（optimal substructure）。

形式化地说，设原问题最优解为 $S^\star$，它由若干子问题解 $S_1, S_2, \ldots, S_k$ 组成。若 $S^\star$ 最优能够推出每个 $S_i$ 都是对应子问题的最优解，则该问题具有最优子结构。证明通常采用反证法：

$$\text{若某个} S_i \text{不是最优，则可替换为更优解，从而改进} S^\star,$$

这与 $S^\star$ 是原问题最优解矛盾。

### 1.3.2 重叠子问题

在问题求解过程中，很多子问题的解会被多次使用。动态规划通过保存子问题的解，把重复计算改成表项访问。

递归计算的总代价通常可理解为：

$$\text{总代价} = \text{不同子问题个数} \times \text{每个子问题的计算代价}.$$

注意：这只是在每个子问题计算代价同阶时的数量级估计，不是一般恒等式；若各子问题代价差异很大，需按具体递推逐项求和。

## 1.4 动态规划算法的设计步骤

1. 分析最优解的结构。
2. 递归地定义最优解的代价。
3. 自底向上地计算最优解的代价并保存，同时记录构造最优解的信息。
4. 根据记录的信息构造最优解。

# 2 矩阵链乘法问题

## 2.1 问题定义

输入矩阵序列

$$\langle A_1, A_2, \ldots, A_n \rangle,$$

其中 $A_i$ 是矩阵。目标是计算

$$A_1 A_2 \cdots A_n$$

的最小代价乘法顺序。

若矩阵 $A$ 为 $p \times q$，矩阵 $B$ 为 $q \times r$，则 $AB$ 的标量乘法次数为

$$pqr.$$

矩阵乘法满足结合律，因此计算顺序不同，结果相同但代价可能不同。例如：

$$A_1 : 10 \times 100, \quad A_2 : 100 \times 5, \quad A_3 : 5 \times 50.$$

若先算 $(A_1 A_2)A_3$：

$$10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 5000 + 2500 = 7500.$$

若先算 $A_1(A_2 A_3)$：

$$100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 25000 + 50000 = 75000.$$

所以不同计算顺序的代价差异很大。

## 2.2 解空间规模

设 $p(n)$ 为计算 $n$ 个矩阵乘积的方法数。其增长与 Catalan 数相关：

$$p(n) = C_{n-1}.$$

Catalan 数的增长量级为

$$C_{n-1} = \Theta\left(\frac{4^n}{n^{3/2}}\right).$$

因此枚举所有加括号方式不可行。

## 2.3 最优子结构

记

$$A_{i\sim j} = A_i A_{i+1} \cdots A_j,$$

并令

$$m[i, j] = \text{计算} A_{i\sim j} \text{ 的最小乘法数}.$$

若 $A_{i\sim j}$ 的最优计算顺序在 $k$ 处断开：

$$A_{i\sim j} = A_{i\sim k} \cdot A_{k+1\sim j},$$

则 $A_{i\sim k}$ 必须按最优顺序计算，$A_{k+1\sim j}$ 也必须按最优顺序计算。否则可以替换其中某一侧为更优计算顺序，从而使整体代价更小，与整体最优矛盾。

## 2.4 状态转移方程

设矩阵 $A_i$ 的维度为

$$p_{i-1} \times p_i.$$

若最后一次乘法在 $k$ 处划分，则：

$$A_{i\sim k} \text{ 的维度为} p_{i-1} \times p_k,$$

$$A_{k+1\sim j} \text{ 的维度为} p_k \times p_j.$$

最后一次相乘的代价为

$$p_{i-1}p_kp_j.$$

因此：

$$m[i,i] = 0,$$

$$m[i,j] = \min_{i\le k<j}\{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\}, \qquad i < j.$$

其中 $k$ 是最后一次乘法的划分点。为了构造最优解，令

$$s[i,j] = \arg\min_{i\le k<j}\{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\}.$$

## 2.5 自底向上计算

矩阵链乘法按链长 $l$ 从短到长计算：

$$l = 2, 3, \ldots, n.$$

每个长度为 $l$ 的区间满足

$$j = i + l - 1.$$

计算 $m[i,j]$ 时，需要枚举

$$k = i, i+1, \ldots, j-1.$$

```
MATRIX-CHAIN-ORDER(p)
    n = length(p) - 1
    for i = 1 to n
        m[i, i] = 0
    for l = 2 to n
```

```
        for i = 1 to n - l + 1
            j = i + l - 1
            m[i, j] = infinity
            for k = i to j - 1
                q = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j]
                if q < m[i, j]
                    m[i, j] = q
                    s[i, j] = k
    return m, s
```

构造最优括号:

```
PRINT-OPTIMAL-PARENS(s, i, j)
    if i == j
        print A_i
    else
        print "("
        PRINT-OPTIMAL-PARENS(s, i, s[i, j])
        print "x"
        PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
        print ")"
```

## 2.6 复杂度

三层循环分别为 $l$、$i$、$k$。粗略上界为:

$$1 \le l \le n, \qquad 1 \le i \le n, \qquad 1 \le k \le n,$$

所以时间复杂度为

$$O(n^3).$$

更细地看，枚举次数为

$$\sum_{l=2}^{n} \sum_{i=1}^{n-l+1} (l-1) = \sum_{l=2}^{n} (n-l+1)(l-1).$$

令 $t = l - 1$，则

$$\sum_{t=1}^{n-1} (n-t)t = n \sum_{t=1}^{n-1} t - \sum_{t=1}^{n-1} t^2 = \Theta(n^3).$$

需要保存 $m$ 和 $s$ 两个二维表，因此空间复杂度为

$$O(n^2).$$

# 3 最长公共子序列问题

## 3.1 基本概念

给定序列 $X = \langle x_1, x_2, \ldots, x_m \rangle$，若存在严格递增下标

$$1 \le i_1 < i_2 < \cdots < i_k \le m,$$

使得

$$Z = \langle x_{i_1}, x_{i_2}, \ldots, x_{i_k} \rangle,$$

则称 $Z$ 是 $X$ 的子序列。

若 $Z$ 同时是 $X$ 与 $Y$ 的子序列，则 $Z$ 是 $X$ 与 $Y$ 的公共子序列。最长公共子序列，即 LCS，是长度最大的公共子序列。

## 3.2 最优子结构

设

$$X_i = \langle x_1, \ldots, x_i \rangle, \qquad Y_j = \langle y_1, \ldots, y_j \rangle.$$

记 $X_i$ 与 $Y_j$ 的 LCS 长度为 $C[i, j]$。

若 $x_i = y_j$，则最后一个字符可以一起进入 LCS：

$$C[i, j] = C[i-1, j-1] + 1.$$

若 $x_i \neq y_j$，则 $x_i$ 与 $y_j$ 不能同时作为同一个末尾匹配字符。最优解来自两个子问题之一：

$$C[i, j] = \max\{C[i-1, j], C[i, j-1]\}.$$

## 3.3 递归方程

完整递推为：

$$C[i,j] = \begin{cases} 0, & i = 0 \text{ 或} j = 0, \\ C[i-1, j-1] + 1, & i, j > 0 \text{ 且} x_i = y_j, \\ \max\{C[i-1, j], C[i, j-1]\}, & i, j > 0 \text{ 且} x_i \neq y_j. \end{cases}$$

为构造具体 LCS，使用指针表 $B$：

$$B[i,j] = \begin{cases} \nwarrow, & x_i = y_j, \\ \uparrow, & x_i \neq y_j \text{ 且} C[i-1, j] \ge C[i, j-1], \\ \leftarrow, & x_i \neq y_j \text{ 且} C[i-1, j] < C[i, j-1]. \end{cases}$$

## 3.4 算法

```
LCS-LENGTH(X, Y)
    m = length(X)
    n = length(Y)
    for i = 1 to m
        C[i, 0] = 0
    for j = 1 to n
        C[0, j] = 0
    for i = 1 to m
        for j = 1 to n
            if X[i] == Y[j]
                C[i, j] = C[i - 1, j - 1] + 1
                B[i, j] = "diag"
            else if C[i - 1, j] >= C[i, j - 1]
                C[i, j] = C[i - 1, j]
                B[i, j] = "up"
            else
                C[i, j] = C[i, j - 1]
                B[i, j] = "left"
    return C, B
```

打印 LCS:

```
PRINT-LCS(B, X, i, j)
    if i == 0 or j == 0
        return
    if B[i, j] == "diag"
        PRINT-LCS(B, X, i - 1, j - 1)
        print X[i]
    else if B[i, j] == "up"
        PRINT-LCS(B, X, i - 1, j)
    else
        PRINT-LCS(B, X, i, j - 1)
```

## 3.5 复杂度

双层循环计算 $m \times n$ 个表项:

$$\sum_{i=1}^{m}\sum_{j=1}^{n} 1 = mn.$$

每个表项只做常数次比较与赋值，因此时间复杂度为

$$O(mn).$$

表 $C$ 与 $B$ 都是二维表，因此空间复杂度为

$$O(mn).$$

# 4 最长上升子序列

给定整数数组 $A$，要求其中最长严格递增子序列的长度。子序列由数组删除若干元素而来，不改变剩余元素的相对顺序。

例如：

$$A = [10, 9, 2, 5, 3, 7, 101, 18].$$

最长递增子序列可以是

$$[2, 3, 7, 101],$$

长度为 4。

用动态规划可令

$$dp[i] = \text{以}A[i]\text{ 结尾的最长严格递增子序列长度}.$$

若 $j < i$ 且 $A[j] < A[i]$，则 $A[i]$ 可以接在以 $A[j]$ 结尾的递增子序列后：

$$dp[i] = \max_{0 \le j < i,\ A[j] < A[i]} \{dp[j] + 1\}.$$

若不存在这样的 $j$，则

$$dp[i] = 1.$$

答案为

$$\max_{0 \le i < n} dp[i].$$

朴素动态规划需要枚举 $(j, i)$ 对：

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2),$$

因此时间复杂度为 $O(n^2)$，空间复杂度为 $O(n)$。

# 5 0-1 背包问题

## 5.1 问题定义

给定 $n$ 种物品和一个背包。物品 $i$ 的重量为 $w_i$，价值为 $v_i$，背包容量为 $C$。每种物品只能选择完全装入或不装入，一个物品至多装入一次。要求选择装入背包的物品，使总重量不超过容量且总价值最大。

用变量

$$x_i \in \{0, 1\}$$

表示是否选择物品 $i$，则问题可写为：

$$\max \sum_{i=1}^{n} v_i x_i$$

满足

$$\sum_{i=1}^{n} w_i x_i \le C, \qquad x_i \in \{0,1\}.$$

## 5.2 状态定义与最优子结构

令

$$m(i,j) = \text{背包容量为} j\text{，可选物品为} 1,2,\ldots,i \text{ 时的最优价值.}$$

对物品 $i$，最优解只有两种可能：

- 不装物品 $i$：价值为 $m(i-1,j)$；
- 装物品 $i$：要求 $j \ge w_i$，价值为 $m(i-1,j-w_i)+v_i$。

因此子问题规模减少到前 $i-1$ 个物品，具有最优子结构。

## 5.3 递推方程

当容量不足以装入物品 $i$ 时：

$$m(i,j) = m(i-1,j), \qquad 0 \le j < w_i.$$

当容量足够时：

$$m(i,j) = \max\{m(i-1,j),\ m(i-1,j-w_i)+v_i\}, \qquad j \ge w_i.$$

初始值可写为：

$$m(1,j) = \begin{cases} 0, & 0 \le j < w_1, \\ v_1, & j \ge w_1. \end{cases}$$

也常使用

$$m(0,j) = 0, \qquad 0 \le j \le C,$$

再从 $i=1$ 开始递推。

## 5.4 构造最优解

最终最优值为

$$m(n,C).$$

从 $(n,C)$ 反向构造：

$$若m(n,C)=m(n-1,C),\ 则x_n=0.$$

$$否则x_n=1,\ 并转到m(n-1,C-w_n).$$

一般地，从 $(i,j)$ 出发：

$$\begin{cases} x_i=0,\ (i,j)\leftarrow(i-1,j), & m(i,j)=m(i-1,j),\\ x_i=1,\ (i,j)\leftarrow(i-1,j-w_i), & m(i,j)\neq m(i-1,j). \end{cases}$$

## 5.5 算法与复杂度

```
ZERO-ONE-KNAPSACK(w, v, n, C)
    for j = 0 to C
        m[0, j] = 0
    for i = 1 to n
        for j = 0 to C
            if j < w[i]
                m[i, j] = m[i - 1, j]
            else
                m[i, j] = max(m[i - 1, j],
                              m[i - 1, j - w[i]] + v[i])
    return m[n, C]
```

表项数为

$$(n+1)(C+1)=\Theta(nC),$$

每个表项常数时间计算，因此时间复杂度为

$$O(nC),$$

空间复杂度为

$$O(nC).$$

注意：$O(nC)$ 中 $C$ 是容量数值，这种复杂度称为伪多项式复杂度——当 $C$ 以二进制位数 $\lceil\log_2 C\rceil$ 计入输入规模时，$C$ 本身可以是指数级的，因此 0-1 背包没有已知的多项式时间算法；该结论建立在重量、容量可按整数下标枚举的基础上。

## 5.6 PPT 例题

给定：

$$v=\{7,3,6,10,8\},\qquad w=\{5,2,2,6,4\},\qquad C=10.$$

递推方程仍为：

$$m(i,j)=m(i-1,j),\qquad 0\le j<w_i,$$

$$m(i,j)=\max\{m(i-1,j),m(i-1,j-w_i)+v_i\},\qquad j\geq w_i.$$

PPT 表中最终结果为：

$$m(5,10)=19.$$

对应选择为第 $2,3,4$ 个物品：

$$w_2+w_3+w_4=2+2+6=10,$$

$$v_2+v_3+v_4=3+6+10=19.$$

# 6 完全背包问题

完全背包与 0-1 背包的目标相同，但每种物品的数量有无限多个。

状态仍可定义为

$$m(i,j)=\text{容量为}j\text{，可选物品为}1,\ldots,i\text{ 时的最优价值.}$$

若容量不足：

$$m(i,j)=m(i-1,j),\qquad 0\leq j<w_i.$$

若容量足够，需要枚举选择物品 $i$ 的件数 $k$：

$$m(i,j)=\max_{0\leq k\leq\lfloor j/w_i\rfloor}\{m(i-1,j-kw_i)+kv_i\},\qquad j\geq w_i.$$

在 $w_1>0$、$v_1\geq 0$ 的假设下，初始值可写为：

$$m(1,j)=0,\qquad 0\leq j<w_1,$$

$$m(1,j)=\left\lfloor\frac{j}{w_1}\right\rfloor v_1,\qquad j\geq w_1.$$

（若 $w_1\leq 0$ 会出现除零或物品可以无限装入；若 $v_1<0$ 则最优选择是尽量少装该物品，公式不再成立。）

若直接按上述公式枚举 $k$，时间复杂度上界为

$$O\left(nC\cdot\max_i\left\lfloor\frac{C}{w_i}\right\rfloor\right).$$

# 7 状态压缩

当某个状态只依赖上一层或少量前序状态时，可以用较少空间保存必要信息。

以 0-1 背包为例：

$$m(i,j)=\max\{m(i-1,j),m(i-1,j-w_i)+v_i\}.$$

第 $i$ 层只依赖第 $i-1$ 层，因此可用一维数组 $dp[j]$ 表示当前最优值。

0-1 背包的一维更新需要容量从大到小枚举：

$$dp[j] = \max\{dp[j], dp[j - w_i] + v_i\}, \qquad j = C, C-1, \ldots, w_i.$$

从大到小更新是为了避免同一物品在同一轮被重复使用。空间复杂度由

$$O(nC)$$

降为

$$O(C).$$

# 8 最优二叉搜索树

## 8.1 问题定义

二叉搜索树满足：

- 左子树非空时，左子树所有结点值均小于父结点值；
- 右子树非空时，右子树所有结点值均大于父结点值；
- 左右子树也都是二叉搜索树；
- 中序遍历二叉搜索树得到有序序列。

每个关键字的搜索代价等于访问的结点数，也等于关键字所在结点深度加 1：

$$\text{cost}(k) = depth(k) + 1.$$

最优二叉搜索树要最小化搜索期望代价。PPT 强调：

- 最优二叉搜索树不一定是高度最矮的；
- 概率高的关键字不一定出现在根结点。

## 8.2 动态规划递推

设成功搜索概率为 $p_i$，失败搜索概率为 $q_i$。令

$$E(i,j) = \text{包含关键字} k_i, \ldots, k_j \text{ 的最优二叉搜索树期望代价},$$

$$W(i,j) = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t.$$

空子树边界：

$$E(i, i-1) = q_{i-1}, \qquad W(i, i-1) = q_{i-1}.$$

若选择 $r$ 为根，其中 $i \le r \le j$，则左子树为 $i,\dots,r-1$，右子树为 $r+1,\dots,j$。由于所有子树结点深度在挂到根下后都会增加 1，因此要额外加上该区间所有概率之和 $W(i,j)$：

$$E(i,j) = \min_{i \le r \le j} \{E(i,r-1) + E(r+1,j) + W(i,j)\}.$$

同时记录根：

$$Root(i,j) = \arg\min_{i \le r \le j} \{E(i,r-1) + E(r+1,j) + W(i,j)\}.$$

权重递推为：

$$W(i,j) = W(i,j-1) + p_j + q_j.$$

## 8.3 算法

```
OPTIMAL-BST(p, q)
    n = length(p)
    for i = 1 to n + 1
        E[i, i - 1] = q[i - 1]
        W[i, i - 1] = q[i - 1]
    for l = 1 to n
        for i = 1 to n - l + 1
            j = i + l - 1
            E[i, j] = infinity
            W[i, j] = W[i, j - 1] + p[j] + q[j]
            for r = i to j
                t = E[i, r - 1] + E[r + 1, j] + W[i, j]
                if t < E[i, j]
                    E[i, j] = t
                    Root[i, j] = r
    return E, Root
```

三层循环给出时间复杂度：

$$O(n^3),$$

二维表 $E, W, Root$ 给出空间复杂度：

$$O(n^2).$$

# 9 期末相关例题

## 9.1 例题 1：动态规划概念判断

**来源：** 2025 哈尔滨工业大学（深圳）数据结构与算法试题，选择题第 3 题。

**题目：** 关于动态规划的说法，不正确的是：

1. 动态规划算法适用于具有优化子结构的问题。
2. 动态规划算法能解决的问题不能用递归解决。
3. 动态规划算法适用于具有重叠子问题的问题。
4. 动态规划的核心思想是记录并重用子问题的解，以避免重复计算。

**解析**：动态规划的典型应用要求最优子结构与重叠子问题；此外它还可用于计数、判定类问题（这类问题侧重状态分解与递推，不涉及“最优”）。它本质上是把递归中会重复出现的子问题保存起来，常可看作“带记录的递归”或“自底向上的递推”。因此“能用动态规划解决的问题不能用递归解决”是错误的。

若朴素递归子问题树中同一状态 $S$ 出现 $r(S)$ 次，递归会产生

$$\sum_{S} r(S)$$

次求解；动态规划只对每个不同状态求解一次，代价变为

$$\sum_{S} 1.$$

**答案**：不正确的是“动态规划算法能解决的问题不能用递归解决”。

## 9.2 例题 2：01 背包填空

**来源**：2025 秋哈尔滨工业大学（深圳）数据结构与算法期末考试试卷（回忆版），填空题第 8 题。

**题目**：设有 01 背包问题：背包容量为 10，物品重量分别为

$$\{1, 2, 3, 5, 7\},$$

价值分别为

$$\{1, 1, 5, 2, 4\}.$$

求能获得的最大价值。

**解析**：令

$$m(i, j) = \text{只考虑前} i \text{ 个物品，容量为} j \text{ 时的最大价值.}$$

递推为

$$m(i, j) = m(i-1, j), \qquad j < w_i,$$

$$m(i, j) = \max\{m(i-1, j), m(i-1, j-w_i) + v_i\}, \qquad j \geq w_i.$$

可直接枚举关键组合验证最优值。第 3 个和第 5 个物品重量为

$$w_3 + w_5 = 3 + 7 = 10,$$

价值为

$$v_3 + v_5 = 5 + 4 = 9.$$

其他达到或接近容量的组合：

$$w_1 + w_2 + w_3 + w_4 = 1 + 2 + 3 + 5 = 11 > 10,$$

不可行；

$$w_2 + w_3 + w_4 = 2 + 3 + 5 = 10, \qquad v_2 + v_3 + v_4 = 1 + 5 + 2 = 8;$$

$$w_1 + w_3 + w_4 = 1 + 3 + 5 = 9, \qquad v_1 + v_3 + v_4 = 1 + 5 + 2 = 8.$$

因此最大价值为 9。

**答案：**

$$9.$$

## 9.3 例题 3：最长公共子串

**来源：** 26 春数据结构与算法期末，大题第 7 题。

**题目：** 最长公共子串定义为两个字符串中同时出现的最长连续公共片段。给定

$$X[1..m], \qquad Y[1..n],$$

设计算法求最长公共子串，写出递推关系、伪代码和时间复杂度。

**解析：** 最长公共子串要求连续，因此状态应表示“以当前位置结尾”的连续公共长度。令

$$dp[i][j] = \text{以}X[i]\text{ 与}Y[j]\text{ 结尾的最长公共后缀长度.}$$

若

$$X[i] = Y[j],$$

则连续性可以从前一对位置延伸：

$$dp[i][j] = dp[i-1][j-1] + 1.$$

若

$$X[i] \neq Y[j],$$

连续性在此处中断：

$$dp[i][j] = 0.$$

边界为

$$dp[0][j] = 0, \qquad dp[i][0] = 0.$$

答案不是 $dp[m][n]$，而是所有表项的最大值：

$$ans = \max_{1 \le i \le m,\ 1 \le j \le n} dp[i][j].$$

```
LONGEST-COMMON-SUBSTRING(X, Y)
    m = length(X)
    n = length(Y)
    for i = 0 to m
        dp[i][0] = 0
    for j = 0 to n
        dp[0][j] = 0
    ans = 0
    for i = 1 to m
        for j = 1 to n
            if X[i] == Y[j]
                dp[i][j] = dp[i - 1][j - 1] + 1
                ans = max(ans, dp[i][j])
            else
                dp[i][j] = 0
    return ans
```

共有 $mn$ 个状态，每个状态 $O(1)$ 转移：

$$T(m,n) = \sum_{i=1}^{m}\sum_{j=1}^{n} O(1) = O(mn).$$

二维表空间为

$$O(mn).$$

若只保留上一行，空间可降为

$$O(n).$$

**答案**：递推关系如上，时间复杂度 $O(mn)$。

## 9.4 例题 4：最长回文子串

**来源**：2025 秋哈尔滨工业大学（深圳）数据结构与算法期末考试试卷（回忆版），手写代码题第 4 题。

**题目**：计算字符串 $s$ 的最长回文子串长度，构建 $dp$ 数组，写出转移方程和边界条件，并编写代码。

**解析**：令

$$dp[i][j] = \begin{cases} 1, & s[i..j] \text{ 是回文串}, \\ 0, & s[i..j] \text{ 不是回文串}. \end{cases}$$

长度为 1 的子串一定是回文：

$$dp[i][i] = 1.$$

长度为 2 的子串是否回文取决于两个字符是否相等：

$$dp[i][i+1] = 1 \iff s[i] = s[i+1].$$

长度至少为 3 时，$s[i..j]$ 是回文当且仅当两端字符相等，且中间部分也是回文：

$$dp[i][j] = 1 \iff s[i] = s[j] \text{ 且 } dp[i+1][j-1] = 1.$$

可统一写为：

$$dp[i][j] = \begin{cases} 1, & i = j, \\ [s[i] = s[j]], & j = i+1, \\ [s[i] = s[j]] \wedge dp[i+1][j-1], & j - i + 1 \geq 3. \end{cases}$$

答案为

$$ans = \max\{j - i + 1 \mid dp[i][j] = 1\}.$$

```
LONGEST-PALINDROME-SUBSTRING(s, n)
    ans = 0
    for i = 0 to n - 1
        dp[i][i] = true
        ans = 1
    for len = 2 to n
        for i = 0 to n - len
            j = i + len - 1
            if s[i] != s[j]
                dp[i][j] = false
            else if len == 2
                dp[i][j] = true
            else
                dp[i][j] = dp[i + 1][j - 1]
            if dp[i][j]
                ans = max(ans, len)
    return ans
```

状态数为所有区间数量：

$$\sum_{\ell=1}^{n} (n - \ell + 1) = \frac{n(n+1)}{2} = \Theta(n^2).$$

每个状态常数时间转移，所以时间复杂度为

$$O(n^2),$$

空间复杂度为

$$O(n^2).$$

**答案**：使用区间 DP，时间复杂度 $O(n^2)$，空间复杂度 $O(n^2)$。

## 9.5 例题 5：序列计数

**来源**： 2025 哈尔滨工业大学（深圳）数据结构与算法试题，大题第 3 题。

**题目**：给定正整数 $n$，求满足以下条件的序列总数：序列元素不是 1 就是 2；不能出现相邻的 2；序列中所有元素之和为 $n$。要求给出状态定义、递推关系、伪代码和时空复杂度。（本题是计数问题，没有“最优解”，因此不适用“最优子结构”的说法。）

**解析**：因为限制条件与结尾元素有关，需要区分序列最后一个元素。

定义：

$$I(i) = \text{和为}i \text{ 且以1 结尾的合法序列数},$$

$$II(i) = \text{和为}i \text{ 且以2 结尾的合法序列数}.$$

若序列以 1 结尾，则前面可以是任意合法序列，和为 $i-1$：

$$I(i) = I(i-1) + II(i-1).$$

若序列以 2 结尾，则前一个元素不能是 2，所以前缀必须以 1 结尾，和为 $i-2$：

$$II(i) = I(i-2).$$

边界：

$$I(0) = 1, \quad II(0) = 0, \quad I(1) = 1, \quad II(1) = 0.$$

说明：按状态定义，和为 0 且以 1 结尾的序列数应为 0；这里设 $I(0) = 1$ 是哨兵约定，用来使递推 $I(1) = I(0) + II(0)$ 成立，必须与 $II(0) = 0$ 配合使用，否则状态语义与边界冲突。答案为

$$F(i) = I(i) + II(i).$$

```
COUNT-SEQUENCES(n)
    if n == 0 return 1
    if n == 1 return 1
    I[0] = 1
    II[0] = 0
    I[1] = 1
    II[1] = 0
    for i = 2 to n
        I[i] = I[i - 1] + II[i - 1]
        II[i] = I[i - 2]
    return I[n] + II[n]
```

循环执行 $n-1$ 次，每次常数时间：

$$T(n) = \sum_{i=2}^{n} O(1) = O(n).$$

两个长度为 $n+1$ 的数组给出空间复杂度：

$$O(n).$$

由于当前状态只依赖 $I(i-1)$、$II(i-1)$、$I(i-2)$，可压缩为常数空间：

$$S(n) = O(1).$$

**答案**：递推为

$$I(i) = I(i-1) + II(i-1), \qquad II(i) = I(i-2),$$

时间复杂度 $O(n)$，数组实现空间 $O(n)$，压缩后空间 $O(1)$。